

UNIVERSIDAD NACIONAL DEL ALTIPLANO – PUNO

Facultad de Ingeniería Mecánica Eléctrica, Electrónica y Sistemas

Escuela Profesional de Ingeniería de Sistemas

ALGORITMOS, TIPOS ABSTRACTOS DE DATOS, CORRECCIÓN Y MODELO RAM

Trabajo académico de la Semana 1

Curso: Algoritmos y Estructuras de Datos – SIS210

Docente: ZANABRIA GALVEZ ALDO HERNAN

Integrantes: YANA MENDOZA CARLOS BENEDICTO
BELIZARIO YANA DAVID VICTOR
CURASI ZEVALLOS HANDDY RONALD

Semestre académico: 2026–II

Repositorio GitHub del trabajo

github.com/handdycurasi-crypto/SIS210-Semana1

Puno, Perú

2026

Índice

1	Introducción	1
2	Objetivos	1
2.1	Objetivo general	1
2.2	Objetivos específicos	1
3	Marco teórico	3
3.1	Núcleo 1: concepto de algoritmo y resolución de problemas	3
3.2	Núcleo 2: entrada, salida y especificación del problema	4
3.3	Núcleo 3: características y propiedades de un algoritmo	5
3.3.1	Finitud	5
3.3.2	Definición precisa	6
3.3.3	Efectividad	6
3.3.4	Relación entre las cinco propiedades	7
3.4	Núcleo 4: tipos de datos primitivos y representación	8
3.5	Núcleo 5: fundamentos de los tipos abstractos de datos	8
3.6	Núcleo 6: diferencia entre interfaz e implementación	9
3.7	Núcleo 7: ejemplos fundamentales de TAD	10
3.8	Núcleo 8: especificación y corrección de algoritmos	11
3.9	Núcleo 9: corrección parcial, corrección total y terminación	12
3.10	Núcleo 10: lógica de Hoare, precondition y postcondition	13
3.11	Núcleo 11: invariantes de ciclo e inducción	14
3.12	Núcleo 12: costo computacional, modelo RAM y crecimiento	15
3.12.1	Motivación del análisis	15
3.12.2	Modelo RAM	16
3.12.3	Operaciones elementales	16
3.12.4	Funciones de crecimiento	17
3.12.5	Relación introductoria con el análisis asintótico	18
4	Desarrollo práctico	20
4.1	Ficha diagnóstica razonada	20
5	Implementación del algoritmo para encontrar el máximo	20
5.1	Idea de solución	20
5.2	Implementación en Python	21
5.2.1	Explicación por bloques	22
5.3	Implementación en C++	22

5.3.1	Explicación por bloques	23
6	Análisis de corrección de encontrar_maximo	24
6.1	Explicación comprensible	24
6.2	Formalización del problema	24
6.3	Invariante y demostración	24
6.3.1	Inicio	24
6.3.2	Preservación	25
6.3.3	Finalización	25
6.3.4	Conclusión de la demostración	25
7	Análisis mediante el modelo RAM	25
7.1	Criterio adoptado	25
7.2	Dos ciclos anidados	25
7.2.1	Por qué el cuerpo se ejecuta n^2 veces	26
7.2.2	Conteo por categorías	26
7.3	Conteo de encontrar_maximo	28
8	Resultados	29
9	Comparación y discusión	29
10	Dificultades encontradas y aprendizaje	30
11	Aspectos que resultaron interesantes	30
12	Conclusiones	31
13	Referencias bibliográficas	32
A	Evidencia de ejecución	34
B	Preguntas probables para la sustentación	34
C	Conceptos prioritarios para estudiar	35

1. Introducción

Los algoritmos y las estructuras de datos constituyen una base para expresar soluciones computacionales de manera ordenada, verificable y eficiente. Antes de estudiar estructuras específicas o técnicas avanzadas de análisis, resulta necesario comprender qué condiciones hacen que un procedimiento pueda considerarse algoritmo, cómo se separa una operación abstracta de su representación concreta y de qué manera se razona sobre la corrección y el costo de una solución. El material oficial de la Semana 1 organiza precisamente estos fundamentos y los relaciona mediante un ejercicio sencillo pero representativo: encontrar el valor máximo de un arreglo (Zanabria Gálvez, 2026; Cormen et al., 2022; Sedgewick y Wayne, 2011).

El presente trabajo amplía esos contenidos sin apartarse del alcance introductorio de la unidad. El marco teórico se organiza en doce núcleos articulados: concepto de algoritmo; entrada y salida; propiedades; tipos primitivos; fundamentos de los TAD; interfaz e implementación; ejemplos de TAD; corrección; corrección parcial y total; lógica de Hoare; invariantes; y costo computacional con modelo RAM. La teoría se conecta con las implementaciones ya verificadas en Python y C++, con la prueba común de 24 elementos y con la comparación entre un recorrido lineal y dos ciclos anidados.

2. Objetivos

2.1. Objetivo general

Analizar de manera fundamentada los conceptos introductorios de algoritmos, tipos abstractos de datos, corrección y costo computacional, aplicándolos a la implementación y verificación del algoritmo que encuentra el máximo de un arreglo en Python y C++.

2.2. Objetivos específicos

- Explicar el concepto de algoritmo y sus propiedades de entrada, salida, finitud, definición precisa y efectividad.
- Diferenciar los tipos de datos primitivos de los TAD y distinguir la interfaz abstracta de su implementación.
- Formular la especificación de `encontrar_maximo` mediante precondition, postcondición y tripleta de Hoare.
- Demostrar la corrección del recorrido usando un invariante y las etapas de inicio, preservación y finalización.

- Contar operaciones elementales con un criterio RAM explícito y comparar crecimientos lineal y cuadrático.
- Ejecutar las implementaciones de Python y C++ y registrar evidencia verificable de sus resultados.

3. Marco teórico

El marco se divide en doce núcleos para que la exposición grupal conserve una secuencia lógica y cada integrante pueda dominar un bloque coherente. La distribución propuesta no divide artificialmente el tema: los primeros cuatro núcleos establecen qué es un algoritmo y con qué datos trabaja; los cuatro siguientes estudian la abstracción y la corrección; y los cuatro últimos formalizan garantías y costo. Los límites entre bloques sirven para organizar la sustentación, pero las relaciones entre todos ellos se explican de forma explícita (Zanabria Gálvez, 2026; Cormen et al., 2022; Wirth, 1976).

Tabla 1: Organización de los doce núcleos para la sustentación grupal

Integrante	Núcleos	Dominio principal
Carlos Bene- dicto Yana Mendoza	1–4	Algoritmo, especificación, propiedades y tipos primitivos.
David Victor Belizario Yana	5–8	TAD, interfaz, implementaciones, ejemplos y corrección.
Handdy Ronald Curasi Zevallos	9–12	Corrección parcial y total, Hoare, invariantes y modelo RAM.

3.1. Núcleo 1: concepto de algoritmo y resolución de problemas

Un algoritmo es un procedimiento computacional bien definido que recibe ciertos valores, llamados entrada, y produce otros valores, llamados salida, mediante una secuencia finita de pasos. Esta definición une tres ideas: existe un problema previamente delimitado, hay una transformación que conduce desde los datos disponibles hasta el resultado y cada paso posee un significado operacional. Por ello, un algoritmo no es una lista arbitraria de instrucciones, sino una descripción organizada de cómo resolver todas las instancias que satisfacen las condiciones del problema (Zanabria Gálvez, 2026; Cormen et al., 2022; Knuth, 1997).

Conviene diferenciar algoritmo y programa. El algoritmo expresa la estrategia de solución en un nivel que puede ser independiente de un lenguaje; el programa es una realización concreta escrita con la sintaxis y los tipos de Python, C++ u otro lenguaje. El mismo recorrido para buscar un máximo puede programarse con un `for`, un `while`, iteradores o funciones de biblioteca, y estas versiones pueden implementar la misma idea esencial. Esta distinción permite discutir la solución antes de quedar condicionados por detalles sintácticos (Cormen et al., 2022; Sedgewick y Wayne, 2011; Skiena, 2020).

También debe diferenciarse un algoritmo de una descripción cotidiana incompleta. La indicación “ordenar los números” señala una meta, pero no determina los pasos para

alcanzarla; en cambio, comparar elementos y efectuar intercambios bajo reglas establecidas sí describe un procedimiento. En una receta ocurre algo parecido: si se indican ingredientes, orden de acciones, condiciones y finalización, puede reconocerse una estructura algorítmica, aunque la computación formal exige eliminar ambigüedades que una persona resolvería por experiencia (Zanabria Gálvez, 2026; Cormen et al., 2022; Goodrich et al., 2013).

El diseño algorítmico parte de identificar con precisión qué se conoce, qué debe obtenerse y qué restricciones existen. Para el problema de este informe, se conoce un arreglo no vacío de valores comparables y se desea devolver uno de ellos que sea mayor o igual que todos los demás. A partir de esa especificación se propone mantener un máximo provisional y actualizarlo al encontrar un elemento mayor. La estrategia es sencilla, pero permite conectar especificación, corrección y eficiencia en un solo ejemplo (Cormen et al., 2022; Goodrich et al., 2011; Skiena, 2020).

La generalidad es otra consecuencia importante del concepto. El algoritmo para hallar el máximo no se diseña únicamente para el arreglo utilizado en la prueba, sino para cualquier arreglo que cumpla la precondition. Los 24 elementos permiten observar una ejecución concreta, mientras la definición y la demostración se formulan con n para abarcar una familia ilimitada de instancias. Esta separación entre ejemplo e instancia general evita confundir una prueba exitosa con una garantía universal (Aho et al., 1974; Cormen et al., 2022; Knuth, 1997).

Un algoritmo tampoco queda definido por la velocidad de una ejecución particular. Dos programas pueden implementar el mismo procedimiento en lenguajes o equipos diferentes y tardar tiempos físicos distintos, aun cuando realicen el mismo patrón lógico de comparaciones. Por ello, primero se identifica la estrategia y después se estudian su corrección y su costo bajo un modelo explícito (Aho et al., 1974; Sedgewick y Wayne, 2011; Skiena, 2020).

3.2. Núcleo 2: entrada, salida y especificación del problema

La entrada está formada por los valores que una instancia entrega al algoritmo. No se limita a aquello que una persona escribe por teclado: puede provenir de un archivo, una estructura en memoria, un sensor o la llamada de otra función. Lo importante es que su dominio esté especificado. En `encontrar_maximo`, la entrada es una secuencia finita y no vacía de valores que admiten comparación; su tamaño se representa mediante n (Zanabria Gálvez, 2026; Cormen et al., 2022; Goodrich et al., 2013).

La salida es el resultado que debe satisfacer la condición establecida por el problema. Para una misma entrada pueden existir problemas con una salida única o con varias respuestas válidas; por ejemplo, el valor máximo es único como valor aunque aparezca en varias posiciones. Definir la salida evita confundir lo que el procedimiento calcula con información auxiliar como el índice del ciclo o el máximo provisional. En este trabajo, la

salida requerida es un valor m perteneciente al arreglo tal que todo elemento es menor o igual que m (Cormen et al., 2022; Hoare, 1969; Goodrich et al., 2011).

La relación entrada-salida constituye el contrato esencial de la solución. Una implementación puede imprimir mensajes adicionales durante una prueba, pero su función principal debe preservar esa relación. Por ello, las versiones desarrolladas devuelven el máximo y el programa de prueba se ocupa separadamente de mostrarlo y contrastarlo con una función estándar. Separar cálculo y presentación facilita reutilizar la función y razonar sobre ella (Liskov y Zilles, 1974; Sedgewick y Wayne, 2011; Goodrich et al., 2013).

Definir el dominio de entrada permite decidir qué casos pertenecen al problema. En esta práctica se incluyen valores positivos, cero y negativos; la posición del máximo no está fijada y podrían existir valores repetidos. En cambio, un arreglo vacío queda fuera del dominio porque no contiene un valor que pueda satisfacer la salida. Esta decisión no es un detalle de programación, sino parte de la especificación que después aparece como precondition (Hoare, 1969; Apt et al., 2009; Cormen et al., 2022).

La salida puede expresarse informalmente como “el mayor valor”, pero para demostrar corrección conviene precisar dos condiciones: el resultado pertenece al arreglo y ningún elemento lo supera. La primera impide devolver un número ajeno a los datos; la segunda establece su maximalidad. Esta formulación muestra cómo una frase cotidiana se transforma en una postcondición verificable (Hoare, 1969; Apt et al., 2009; Dijkstra, 1975).

3.3. Núcleo 3: características y propiedades de un algoritmo

3.3.1. Finitud

La finitud exige que el procedimiento termine después de un número finito de pasos para cada entrada admitida. No basta con que cada instrucción sea comprensible si el proceso puede continuar indefinidamente. En un recorrido de arreglo, la finitud se justifica mostrando que el índice avanza y que existe un límite fijo dado por n . Tras procesar el último elemento, la condición del ciclo deja de cumplirse y se obtiene la salida (Zanabria Gálvez, 2026; Cormen et al., 2022; Hoare, 1969).

La finitud depende tanto de la estructura del algoritmo como de sus condiciones de entrada. Un ciclo que disminuye una variable positiva en una unidad termina si esa variable pertenece a los enteros no negativos, pero la misma afirmación puede fallar si no se fija el dominio o si el cuerpo no garantiza el avance. En las demostraciones formales suele utilizarse una cantidad variante que disminuye y está acotada inferiormente; para `encontrar_maximo`, la cantidad $n - i$ disminuye en cada iteración (Hoare, 1969; Dijkstra, 1975; Cormen et al., 2022).

Finitud no significa rapidez. Un algoritmo puede terminar siempre y, sin embargo,

requerir una cantidad impráctica de operaciones cuando la entrada crece. Esta observación separa dos preguntas: “¿terminará y dará la respuesta correcta?” y “¿cuántos recursos necesitará?”. La primera pertenece a la corrección total; la segunda, al análisis de costo. Ambas son necesarias para evaluar una solución, pero no deben confundirse (Cormen et al., 2022; Skiena, 2020; Sedgewick y Wayne, 2011).

3.3.2. Definición precisa

La definición precisa establece que cada paso debe interpretarse sin ambigüedad. Expresiones como “elegir un número grande” o “repetir varias veces” no son suficientes si no se determina qué significa grande o cuántas repeticiones corresponden. En un algoritmo formal se especifican las condiciones, los datos afectados y el orden de ejecución. Esta precisión permite que distintas personas o máquinas apliquen el procedimiento de la misma manera (Zanabria Gálvez, 2026; Cormen et al., 2022; Knuth, 1997).

La precisión también abarca casos límite. Decir que se toma el primer elemento como máximo provisional presupone que existe un primer elemento; por eso la entrada no puede estar vacía. Asimismo, debe aclararse si los valores pueden ser negativos, si están repetidos y qué tipo de comparación se utiliza. Las implementaciones del proyecto aceptan enteros negativos y repetidos, pero rechazan explícitamente un arreglo vacío, lo que hace visible la condición que el algoritmo original supone (Goodrich et al., 2013, 2011; Sedgewick y Wayne, 2011).

Un pseudocódigo puede ser preciso sin copiar todos los detalles de un lenguaje real. Su propósito es expresar las operaciones relevantes y ocultar aspectos que no cambian la estrategia, como la forma exacta de declarar un vector. No obstante, cuando se cuenta el costo, algunos de esos detalles vuelven a importar: un `for` contiene de manera implícita inicialización, verificación e incremento. Por eso el nivel de abstracción debe declararse antes de producir una fórmula exacta (Cormen et al., 2022; Knuth, 1997; Cook y Reckhow, 1973).

3.3.3. Efectividad

La efectividad exige que cada operación sea suficientemente elemental para poder realizarse de manera exacta y en tiempo finito. Asignar un valor, sumar dos números de tamaño acotado, comparar dos elementos y acceder a una posición de memoria son ejemplos usuales en el modelo introductorio. En cambio, una instrucción como “encontrar mágicamente la mejor solución” no constituye un paso efectivo porque oculta el trabajo que debe efectuarse (Zanabria Gálvez, 2026; Cook y Reckhow, 1973; Cormen et al., 2022).

La efectividad relaciona la descripción abstracta con la posibilidad de implementación. Una operación puede presentarse como una unidad en un nivel y descomponerse en varias operaciones en otro. Por ejemplo, obtener el máximo mediante una función de biblioteca es

efectivo, pero para analizar la estrategia interesa abrir esa operación y reconocer el recorrido y las comparaciones que realiza. Esta elección de granularidad es fundamental para construir un conteo consistente (Sedgewick y Wayne, 2011; Skiena, 2020; Knuth, 1997).

También existen límites prácticos al supuesto de operaciones constantes. La suma de enteros almacenados en una palabra de máquina suele tratarse como constante, pero trabajar con enteros de longitud arbitraria puede hacer que el costo dependa de la cantidad de dígitos. En la Semana 1 se adopta deliberadamente el modelo RAM elemental y valores que caben en una palabra, porque el objetivo es aprender a contar y comparar crecimientos sin introducir todavía modelos más complejos (Cook y Reckhow, 1973; Cormen et al., 2022; Zanabria Gálvez, 2026).

3.3.4. Relación entre las cinco propiedades

Entrada, salida, finitud, definición precisa y efectividad se complementan. La entrada delimita las instancias; la salida fija el resultado esperado; la precisión elimina interpretaciones alternativas; la efectividad asegura que los pasos puedan realizarse; y la finitud garantiza que el proceso llegue a entregar la salida. Omitir una sola propiedad puede convertir una idea intuitiva en una especificación insuficiente o en un proceso no ejecutable (Zanabria Gálvez, 2026; Cormen et al., 2022; Knuth, 1997).

Estas propiedades no demuestran por sí solas que el resultado sea correcto. Un procedimiento puede ser finito, preciso y efectivo, pero devolver una respuesta equivocada. Por ello se necesita relacionar los estados del programa con afirmaciones lógicas y comprobar que cada paso conserva la verdad necesaria. La teoría de corrección amplía así la definición operacional del algoritmo con una garantía sobre su comportamiento (Hoare, 1969; Dijkstra, 1975; Cormen et al., 2022).

Las propiedades pueden utilizarse como una lista de control al revisar una solución. Se pregunta si todas las entradas válidas están descritas, si la salida tiene una condición comprobable, si cada instrucción posee una interpretación única, si las operaciones son realizables y si existe un argumento de terminación. Aplicar estas preguntas al máximo revela de inmediato por qué se requiere un arreglo no vacío y por qué el índice debe aumentar en cada vuelta (Zanabria Gálvez, 2026; Aho et al., 1974; Cormen et al., 2022).

También es útil distinguir una propiedad del algoritmo de una propiedad de una sola ejecución. Observar que el programa terminó para $n = 24$ no demuestra finitud para todo n ; la garantía procede de la estructura del ciclo y del avance acotado del índice. Del mismo modo, acertar el máximo de un arreglo no prueba corrección general. Esta diferencia justifica combinar experimentación con razonamiento formal (Apt et al., 2009; Hoare, 1969; Cormen et al., 2022).

3.4. Núcleo 4: tipos de datos primitivos y representación

Los tipos primitivos son categorías básicas provistas directamente por un lenguaje o por su modelo de ejecución. Entre ellos suelen encontrarse enteros, números de punto flotante, booleanos y caracteres. Un tipo determina qué valores puede almacenar una variable y qué operaciones elementales pueden aplicarse sobre ellos. Por ejemplo, un entero admite suma y comparación, mientras que un booleano representa condiciones verdaderas o falsas (Goodrich et al., 2013, 2011; Sedgewick y Wayne, 2011).

La denominación “primitivo” depende del lenguaje. C++ distingue tipos fundamentales como `int`, `double`, `char` y `bool`, con representaciones vinculadas a la plataforma. Python ofrece objetos integrados como `int`, `float`, `bool` y `str`; aunque internamente sean objetos, funcionan como bloques básicos para el programador. Lo relevante para este nivel es que el lenguaje ya define sus valores y operaciones principales (Goodrich et al., 2013, 2011; Python Software Foundation, 2026).

Los tipos primitivos no expresan por sí solos políticas complejas de organización. Un conjunto de enteros puede almacenarse en un arreglo, pero la idea de una pila incluye además una regla: el último elemento insertado es el primero que se retira. Esa regla de comportamiento no pertenece al entero ni al arreglo aislado, sino a una abstracción definida mediante operaciones y restricciones. Esta necesidad conduce a los tipos abstractos de datos (Liskov y Zilles, 1974; Sedgewick y Wayne, 2011; Skiena, 2020).

La elección del tipo influye en qué entradas son representables. Un entero de C++ posee un rango finito dependiente de su tipo concreto, mientras que el entero de Python puede aumentar su tamaño interno según el valor disponible en memoria. Para la prueba se eligieron números pequeños, de modo que la comparación sea directa en ambos lenguajes y el análisis introductorio pueda tratarlos como valores de una palabra sin introducir costos por longitud numérica (Goodrich et al., 2011, 2013; Python Software Foundation, 2026).

El tipo también protege operaciones inválidas y comunica intención. Declarar un vector de enteros señala que sus elementos comparten un dominio comparable; usar un booleano para la verificación expresa que solo interesa saber si dos resultados coinciden. Antes de construir un TAD complejo, comprender estas garantías básicas permite reconocer qué aporta el lenguaje y qué reglas deben ser diseñadas por el programador (Wirth, 1976; Goodrich et al., 2011; Sedgewick y Wayne, 2011).

3.5. Núcleo 5: fundamentos de los tipos abstractos de datos

Un tipo abstracto de datos (TAD) especifica un dominio de valores y un conjunto de operaciones observables sin imponer una representación interna única. El usuario del TAD conoce qué operaciones puede solicitar, qué condiciones requieren y qué resultados

producen; la estructura concreta usada para almacenarlo permanece separada. Esta forma de abstracción fue desarrollada como un mecanismo para organizar programas alrededor de contratos estables (Liskov y Zilles, 1974; Zanabria Gálvez, 2026; Sedgewick y Wayne, 2011).

El TAD no debe entenderse solamente como “un grupo de datos”. Incluye comportamiento. Una pila se caracteriza por operaciones como crear, apilar, consultar el tope, desapilar y comprobar si está vacía; una cola, por insertar al final y retirar desde el frente. Dos colecciones pueden contener exactamente los mismos elementos y diferir como TAD por las reglas que determinan el próximo elemento accesible (Sedgewick y Wayne, 2011; Goodrich et al., 2013, 2011).

La abstracción reduce la cantidad de detalles que deben mantenerse simultáneamente. Quien usa una pila puede razonar con las operaciones **apilar** y **desapilar**, mientras quien la implementa se ocupa de índices, nodos o ampliaciones de capacidad. Esta división favorece la modularidad y permite reemplazar una representación siempre que se conserve el comportamiento prometido por la interfaz (Liskov y Zilles, 1974; Sedgewick y Wayne, 2011; Skiena, 2020).

La especificación de un TAD puede entenderse como un conjunto de valores válidos más reglas sobre sus operaciones. Por ejemplo, si se apila x en una pila y no ocurre otra modificación, consultar el tope debe devolver x . Esta regla describe comportamiento sin indicar si x se guardó en una posición de arreglo o en un nodo. La abstracción, por tanto, conserva lo esencial y pospone decisiones que no necesita conocer el usuario (Liskov y Zilles, 1974; Wirth, 1976; Goodrich et al., 2013).

Los TAD conectan algoritmos y estructuras de datos. Un algoritmo solicita operaciones abstractas, pero el costo real de atenderlas depende de la representación elegida. Esta relación explica por qué ambas materias se estudian juntas: la abstracción permite razonar con claridad y la implementación permite evaluar recursos. Confundirlas impediría cambiar una representación sin reescribir toda la solución (Wirth, 1976; Aho et al., 1974; Sedgewick y Wayne, 2011).

3.6. Núcleo 6: diferencia entre interfaz e implementación

La interfaz describe qué puede hacerse con el TAD. Incluye nombres de operaciones, parámetros, resultados y condiciones relevantes. La implementación describe cómo se almacenan los valores y cómo se ejecuta cada operación. Por ejemplo, una interfaz de pila puede ofrecer **push**, **pop** y **top**; una implementación puede usar un arreglo redimensionable y otra una lista enlazada (Liskov y Zilles, 1974; Goodrich et al., 2011; Sedgewick y Wayne, 2011).

Cambiar la implementación puede modificar el consumo de memoria, el costo de una

operación o la forma de gestionar capacidad, pero no debe alterar el significado observable de la interfaz. Si `pop` deja de retirar el último elemento insertado, no se trata simplemente de otra implementación: se ha violado el contrato de la pila. Por ello la independencia entre interfaz e implementación no elimina obligaciones; las concentra en una especificación común (Liskov y Zilles, 1974; Goodrich et al., 2013; Skiena, 2020).

La separación puede observarse en lenguajes distintos. Python ofrece listas con operaciones integradas que permiten implementar una pila; C++ ofrece contenedores y adaptadores como `vector` o `stack`. Sin embargo, el concepto LIFO es independiente de la sintaxis de cualquiera de esos lenguajes. Estudiar primero el TAD permite reconocer la misma estructura conceptual aunque cambien las herramientas concretas (Python Software Foundation, 2026; Goodrich et al., 2011; Sedgewick y Wayne, 2011).

La ocultación de detalles reduce el acoplamiento entre módulos. Si el código cliente accede directamente a posiciones internas, cualquier cambio de representación puede romperlo; si utiliza únicamente la interfaz, la implementación conserva libertad para mejorar. Esta idea es relevante incluso en un ejercicio pequeño: la función de máximo necesita recibir una colección recorrible y comparable, pero no necesita conocer cómo se administra su memoria internamente (Liskov y Zilles, 1974; Goodrich et al., 2011; Wirth, 1976).

Una interfaz bien definida incluye casos de error. Una operación `desapilar` debe indicar qué ocurre cuando la pila está vacía; de modo semejante, la función del máximo debe declarar que requiere al menos un elemento. Explicitar estas condiciones evita que la implementación tome decisiones silenciosas y permite que precondiciones y excepciones reflejen el mismo contrato (Liskov y Zilles, 1974; Apt et al., 2009; Goodrich et al., 2013).

Tabla 2: Diferencia entre abstracción e implementación

Aspecto	TAD / interfaz	Implementación concreta
Pregunta central	¿Qué operaciones y reglas ofrece?	¿Cómo se almacenan y ejecutan?
Visibilidad	Contrato observable por el usuario	Detalles internos ocultables
Ejemplo de pila	Contrato observable por el usuario: apilar, desapilar, tope, está vacía	Detalles internos ocultables: arreglo dinámico o lista enlazada
Posibilidad de cambio	Debe conservar su significado	Puede sustituirse por otra representación

3.7. Núcleo 7: ejemplos fundamentales de TAD

La pila sigue la política LIFO: el último elemento que entra es el primero que sale. Resulta útil para deshacer acciones, evaluar expresiones o gestionar llamadas. La cola sigue la política FIFO: el primero que entra es el primero que sale, como ocurre en la atención de

solicitudes. En ambos casos, la política de acceso define la abstracción, no el material con el que se construye (Sedgewick y Wayne, 2011; Goodrich et al., 2013, 2011).

Una lista describe una secuencia ordenada de elementos con operaciones para insertar, eliminar, consultar o recorrer posiciones. Puede implementarse mediante un arreglo contiguo o mediante nodos enlazados. Estas representaciones ofrecen comportamientos equivalentes desde la interfaz, pero presentan costos distintos: acceder por índice es directo en un arreglo, mientras insertar cerca de un nodo conocido puede ser conveniente en una lista enlazada (Goodrich et al., 2013, 2011; Skiena, 2020).

El conjunto modela una colección sin duplicados y suele incluir insertar, eliminar y consultar pertenencia. El diccionario o mapa asocia claves con valores y permite buscar mediante la clave. Sus implementaciones pueden basarse en tablas hash o árboles, decisión que afecta el rendimiento sin cambiar el contrato general. Estos ejemplos muestran que TAD y análisis de eficiencia se relacionan, aunque conceptualmente deban distinguirse (Cormen et al., 2022; Sedgewick y Wayne, 2011; Skiena, 2020).

Incluso el arreglo usado en el ejercicio puede analizarse en dos niveles. Como secuencia indexada ofrece acceso a una posición y conocimiento de su longitud; en Python se representa aquí mediante una lista y en C++ mediante `std::vector<int>`. El algoritmo requiere solamente recorrido, comparación y acceso a los elementos, de modo que su idea no depende de la representación exacta mientras esas operaciones estén disponibles (Goodrich et al., 2013, 2011; Python Software Foundation, 2026).

Comparar los ejemplos revela que una colección no se identifica únicamente por sus elementos. Pila y cola podrían contener la misma secuencia, pero difieren en cuál elemento puede retirarse primero; lista y conjunto difieren en orden y duplicados; diccionario agrega una asociación entre clave y valor. Las operaciones permitidas y sus reglas observables son las que determinan el TAD (Sedgewick y Wayne, 2011; Goodrich et al., 2013; Wirth, 1976).

Para seleccionar un TAD conviene partir de las necesidades del problema y no de la estructura conocida por el programador. Si se requiere atender elementos en orden de llegada, una cola expresa directamente la política; si se requiere revertir acciones, una pila es más natural. La elección conceptual precede a decidir entre arreglo, lista enlazada u otra representación concreta (Skiena, 2020; Aho et al., 1974; Goodrich et al., 2011).

3.8. Núcleo 8: especificación y corrección de algoritmos

Un algoritmo es correcto respecto de una especificación cuando produce el resultado exigido para toda entrada que satisface la precondition. La frase “respecto de una especificación” es indispensable: no puede evaluarse la corrección sin indicar qué entradas se aceptan y qué relación debe cumplir la salida. Probar algunos ejemplos aporta evidencia, pero no

reemplaza una demostración que abarque todas las entradas válidas (Hoare, 1969; Cormen et al., 2022; Zanabria Gálvez, 2026).

Las pruebas experimentales y la demostración formal cumplen funciones complementarias. Ejecutar un programa ayuda a detectar errores concretos y comprueba que la implementación funciona en el entorno utilizado. La demostración estudia la estructura lógica del procedimiento y explica por qué no existe una entrada válida que contradiga la postcondición. En este trabajo se realizan ambas: se prueba un arreglo de 24 elementos y se desarrolla un invariante para el caso general (Hoare, 1969; Dijkstra, 1975; Cormen et al., 2022).

La corrección tampoco implica eficiencia. Un método que compare todas las parejas posibles para obtener cierta información puede producir siempre la respuesta correcta y, sin embargo, realizar trabajo innecesario frente a otro método lineal. La corrección responde si la solución cumple; el costo computacional permite comparar cuánto trabajo requiere. Conservar esta diferencia evita rechazar una prueba correcta porque el algoritmo es lento o aceptar un algoritmo rápido que entrega respuestas erróneas (Cormen et al., 2022; Skiena, 2020; Sedgewick y Wayne, 2011).

La corrección es relativa a la especificación, no a la intención no escrita del autor. Si la postcondición solo exigiera devolver un elemento del arreglo, casi cualquier posición cumpliría, pero no se habría formalizado el máximo. Una especificación débil puede hacer que un programa incorrecto parezca correcto; una especificación contradictoria puede ser imposible de satisfacer. Revisar el contrato es, por ello, parte de la verificación (Hoare, 1969; Apt et al., 2009; Dijkstra, 1975).

Las pruebas con casos borde siguen siendo valiosas aunque no constituyan una demostración. Conviene incluir un único elemento, números negativos, máximos al principio y al final, y valores repetidos. Estos casos pueden descubrir fallos de implementación; la prueba formal explica después por qué, si se respetan las condiciones, el razonamiento cubre todos ellos (Cormen et al., 2022; Apt et al., 2009; Goodrich et al., 2013).

3.9. Núcleo 9: corrección parcial, corrección total y terminación

La corrección parcial afirma que, si el algoritmo termina desde un estado que satisface la precondición, entonces el estado final satisface la postcondición. La afirmación es condicional y no asegura que la terminación ocurra. Esta forma de corrección permite concentrarse primero en la conservación lógica de las propiedades durante la ejecución (Zanabria Gálvez, 2026; Hoare, 1969; Dijkstra, 1975).

La corrección total añade la garantía de terminación. Para obtenerla se combinan dos componentes: la prueba de corrección parcial y un argumento que demuestra que el

proceso no puede repetirse indefinidamente. En un ciclo sobre n elementos, puede emplearse la cantidad restante $n - i$; es un entero no negativo mientras continúa el ciclo y disminuye estrictamente en cada iteración (Hoare, 1969; Dijkstra, 1975; Cormen et al., 2022).

Un ejemplo ayuda a ver la diferencia. Un ciclo que conserva correctamente un máximo provisional pero olvida incrementar el índice puede mantener verdadera su propiedad lógica y aun así no terminar. Bajo corrección parcial, la frase “si termina, devuelve el máximo” podría mantenerse sin garantizar una respuesta efectiva. La corrección total detecta la insuficiencia porque exige justificar el progreso hasta la condición de salida (Zanabria Gálvez, 2026; Hoare, 1969; Cormen et al., 2022).

La terminación suele justificarse mediante una función variante: una cantidad entera no negativa que disminuye estrictamente cada vez que el ciclo se repite. No basta con decir que “el índice aumenta”, porque debe relacionarse ese aumento con un límite. En la búsqueda del máximo, $n - i$ comienza con $n - 1$, disminuye en uno y llega a cero; por ello solo pueden ocurrir $n - 1$ iteraciones (Apt et al., 2009; Dijkstra, 1975; Cormen et al., 2022).

Separar parcial y total mejora la claridad de la prueba. Primero se demuestra que el estado conservado por el ciclo conduce a la postcondición; después se demuestra que el ciclo alcanza su salida. Esta división permite localizar errores: un invariante equivocado afecta la validez del resultado, mientras una variante que no disminuye afecta la garantía de terminación (Apt et al., 2009; Hoare, 1969; Dijkstra, 1975).

3.10. Núcleo 10: lógica de Hoare, precondition y postcondition

Hoare propuso una notación para relacionar afirmaciones lógicas con instrucciones de programa. Una tripleta $\{P\} S \{Q\}$ contiene una precondition P , un programa o bloque S y una postcondition Q . Su lectura para corrección parcial es: si P es verdadera antes de ejecutar S y S termina, entonces Q será verdadera después (Hoare, 1969; Zanabria Gálvez, 2026; Dijkstra, 1975).

La precondition delimita lo que el algoritmo puede suponer al comenzar. No es una explicación informal del objetivo, sino un predicado sobre la entrada y el estado inicial. Para `encontrar_maximo`, se requiere $n \geq 1$ y que los elementos sean comparables. Si se permitiera $n = 0$, la instrucción que accede a la posición cero carecería de un elemento válido (Hoare, 1969; Cormen et al., 2022; Goodrich et al., 2011).

La postcondition expresa lo que debe ser verdadero al finalizar. Puede escribirse como $m \in A$ y $\forall k \in \{0, \dots, n - 1\} : A[k] \leq m$, donde m es el valor devuelto. La pertenencia impide aceptar un valor externo artificialmente grande; la desigualdad confirma que ningún elemento supera al resultado. Juntas caracterizan el máximo del arreglo (Hoare, 1969; Cormen et al., 2022; Zanabria Gálvez, 2026).

La lógica de Hoare permite componer razonamientos locales. Una asignación transforma las condiciones que deben cumplirse; una selección se analiza por sus ramas; y un ciclo necesita una afirmación que permanezca cierta. Este enfoque convierte el programa en objeto de razonamiento matemático y obliga a declarar supuestos que con frecuencia permanecen ocultos en una prueba informal (Hoare, 1969; Dijkstra, 1975; Liskov y Zilles, 1974).

La tripleta no afirma por sí sola que P o Q sean adecuadas: estas deben describir fielmente el problema. En el máximo, P incluye la existencia de al menos un elemento y Q incluye pertenencia y maximalidad. Una vez fijadas, cada línea del algoritmo puede interpretarse como una transformación de estado que acerca P hacia Q sin perder las propiedades necesarias (Hoare, 1969; Apt et al., 2009; Cormen et al., 2022).

Para una estructura condicional se consideran los dos caminos. Si $A[i]$ supera al máximo anterior, la asignación incorpora el nuevo valor; si no, conservar el anterior es correcto. El análisis de ambas ramas evita razonar solo sobre el caso en que existe actualización y demuestra que la postcondición intermedia se mantiene cualquiera que sea el resultado de la comparación (Hoare, 1969; Dijkstra, 1975; Apt et al., 2009).

3.11. Núcleo 11: invariantes de ciclo e inducción

Un invariante de ciclo es una proposición que se mantiene verdadera antes de cada iteración y que conecta el trabajo ya realizado con la meta final. No significa que todas las variables mantengan el mismo valor; lo invariable es una relación lógica. En la búsqueda del máximo, después de procesar un prefijo, la variable `maximo` representa el máximo de ese prefijo (Cormen et al., 2022; Hoare, 1969; Zanabria Gálvez, 2026).

Para el ciclo que comienza con $i = 1$, el invariante puede formularse así: antes de cada iteración con índice i , `maximo` es el máximo de $A[0 \dots i - 1]$. La afirmación coincide con el avance real del algoritmo: los elementos anteriores a i ya fueron examinados y los que empiezan en i todavía no. Elegir correctamente el intervalo evita errores de uno en los límites (Cormen et al., 2022; Goodrich et al., 2013, 2011).

La demostración tiene tres etapas. En el inicio, antes de la primera iteración, $i = 1$ y el prefijo procesado contiene solamente $A[0]$; como `maximo` recibe $A[0]$, el invariante es verdadero. Este caso base muestra que el ciclo comienza desde un estado coherente, no desde una suposición sin establecer (Cormen et al., 2022; Hoare, 1969; Dijkstra, 1975).

En la preservación se supone verdadero el invariante al inicio de una iteración. Si $A[i] > \text{maximo}$, la asignación reemplaza el máximo anterior por el nuevo elemento; si no, el valor se conserva. En ambos casos, al terminar el cuerpo `maximo` es el máximo de $A[0 \dots i]$. Tras incrementar i , esta es exactamente la afirmación requerida para la siguiente iteración

(Cormen et al., 2022; Hoare, 1969; Zanabria Gálvez, 2026).

En la finalización, el ciclo se detiene cuando $i = n$. El invariante afirma entonces que `maximo` es el máximo de $A[0 \dots n - 1]$, que es todo el arreglo. Por tanto se obtiene la postcondición. Si además se observa que $n - i$ disminuye en uno y no puede ser negativo durante el ciclo, se demuestra terminación y se alcanza la corrección total (Cormen et al., 2022; Hoare, 1969; Dijkstra, 1975).

Los invariantes no se descubren únicamente manipulando símbolos; suelen surgir al preguntar qué significa el estado parcial del programa. En este caso la variable se llama `maximo`, pero su significado preciso cambia con el avance: no es todavía el máximo global, sino el máximo de lo revisado. Expresar ese significado permite explicar de manera comprensible el código y después convertirlo en una prueba formal (Cormen et al., 2022; Skiena, 2020; Hoare, 1969).

El razonamiento del invariante se parece a una inducción. El inicio cumple el papel del caso base; la preservación demuestra que la verdad pasa de un prefijo al siguiente; y la finalización aplica la propiedad al prefijo que ya coincide con el arreglo completo. Esta analogía ayuda a comprender por qué tres ejemplos consecutivos no bastan: la preservación debe formularse para una iteración arbitraria (Cormen et al., 2022; Apt et al., 2009; Hoare, 1969).

Un buen invariante debe ser suficientemente fuerte para implicar la postcondición al finalizar, pero también demostrable desde la inicialización. Decir solamente que `maximo` pertenece al arreglo es verdadero, pero no prueba que sea el mayor; afirmar desde el inicio que ya es el máximo global sería fuerte, pero no está establecido. El máximo del prefijo logra el equilibrio correcto (Apt et al., 2009; Cormen et al., 2022; Dijkstra, 1975).

3.12. Núcleo 12: costo computacional, modelo RAM y crecimiento

3.12.1. Motivación del análisis

Medir solamente segundos puede producir conclusiones dependientes del procesador, compilador, lenguaje, carga del sistema y datos concretos. El análisis teórico busca una descripción más estable contando operaciones en función del tamaño de entrada. No pretende negar la utilidad de medir programas reales; ofrece una base independiente para anticipar cómo cambia el trabajo cuando el problema crece (Zanabria Gálvez, 2026; Cormen et al., 2022; Sedgewick y Wayne, 2011).

El costo computacional puede referirse a tiempo, memoria u otros recursos. En esta introducción se estudia principalmente el tiempo mediante operaciones elementales. Se identifica el tamaño n , se determina cuántas veces se ejecuta cada instrucción y se construye una función $T(n)$. Este procedimiento permite comparar un recorrido único con

dos recorridos anidados sin necesitar que ambos estén escritos en el mismo lenguaje (Cormen et al., 2022; Skiena, 2020; Knuth, 1997).

La elección de entrada relevante depende del problema. Para buscar el máximo, n es el número de elementos. No sería adecuado usar como tamaño el valor numérico del máximo, porque un arreglo de tres enteros puede contener un número muy grande y seguir requiriendo solamente tres posiciones examinadas bajo el modelo adoptado. Definir n correctamente es el primer paso de un análisis coherente (Cormen et al., 2022; Goodrich et al., 2013; Sedgewick y Wayne, 2011).

3.12.2. Modelo RAM

El modelo de acceso aleatorio a memoria, o RAM, representa una máquina idealizada con memoria direccionable y un conjunto de instrucciones básicas. En el enfoque introductorio, asignaciones, accesos, comparaciones y operaciones aritméticas sobre palabras de tamaño adecuado se consideran de costo constante. La formalización de Cook y Reckhow permitió estudiar el tiempo de máquinas de acceso aleatorio de manera matemática (Cook y Reckhow, 1973; Zanabria Gálvez, 2026; Cormen et al., 2022).

“Acceso aleatorio” indica que una posición de memoria puede consultarse directamente mediante su dirección, sin recorrer obligatoriamente todas las posiciones anteriores. Por eso acceder a $A[i]$ se modela como una operación constante. Esta idealización se aproxima al funcionamiento de arreglos en memoria y facilita concentrarse en la cantidad de accesos y operaciones, aunque una computadora real tenga jerarquías de caché y diferencias de latencia (Cook y Reckhow, 1973; Knuth, 1997; Cormen et al., 2022).

El modelo debe aplicarse con un criterio declarado. Se puede contar cada línea de alto nivel como una operación o descomponer expresiones y control de ciclos. Ninguna fórmula exacta es universal si dos análisis agrupan operaciones de manera distinta. La comparación válida exige mantener el mismo criterio dentro del análisis y distinguir entre la función exacta obtenida bajo ese criterio y el orden de crecimiento dominante (Cormen et al., 2022; Sedgewick y Wayne, 2011; Skiena, 2020).

3.12.3. Operaciones elementales

Una asignación almacena un valor en una variable, como $s \leftarrow 0$ o $i \leftarrow i+1$. Aunque la segunda expresión también contiene una suma, el acto de guardar el resultado se cuenta separadamente en el modelo expandido de este informe. Distinguir evaluación y almacenamiento permite explicar de dónde sale cada término y evita ocultar trabajo en una única línea (Zanabria Gálvez, 2026; Cormen et al., 2022; Knuth, 1997).

Una comparación evalúa una relación y produce una condición lógica. En el algoritmo para encontrar el máximo aparecen comparaciones entre datos, $A[i] > \text{maximo}$, y compara-

ciones de control, $i < n$. La primera decide si se actualiza la respuesta; la segunda decide si continúa el ciclo. Si el enunciado solicita solo comparaciones de datos se obtiene $n - 1$; si incluye el control explícito, también se cuentan las evaluaciones de la condición del ciclo (Cormen et al., 2022; Sedgewick y Wayne, 2011; Goodrich et al., 2011).

Las operaciones aritméticas incluyen suma, resta y multiplicación. En $s = s + i \cdot j$ se realiza una multiplicación y una suma antes de asignar el nuevo valor. Los incrementos $i = i + 1$ y $j = j + 1$ añaden otras sumas implícitas. En un análisis de alto nivel, los incrementos del `for` pueden quedar agrupados; en uno expandido deben hacerse visibles y contarse (Zanabria Gálvez, 2026; Cormen et al., 2022; Skiena, 2020).

El acceso a una posición, la llamada a obtener una longitud y el retorno pueden incluirse en modelos más detallados. Este informe concentra la tabla principal en asignaciones, comparaciones, sumas y multiplicaciones, porque son las categorías solicitadas. Los accesos se reconocen conceptualmente, pero no se agregan como una quinta categoría para no mezclar criterios durante la suma total (Cook y Reckhow, 1973; Cormen et al., 2022; Zanabria Gálvez, 2026).

3.12.4. Funciones de crecimiento

Una función constante, como $T(n) = 5$, no aumenta con n dentro del modelo. Puede representar la consulta directa de una posición válida o una secuencia fija de instrucciones. La palabra constante no significa que el tiempo real sea exactamente igual en toda máquina, sino que el número modelado de pasos no depende del tamaño de la entrada (Cormen et al., 2022; Sedgewick y Wayne, 2011; Skiena, 2020).

El crecimiento logarítmico aparece cuando cada paso reduce una parte del problema por un factor, por ejemplo al dividir repetidamente un intervalo a la mitad. Si se duplica n , el número de pasos aumenta aproximadamente en una unidad para base dos. Esta clase se menciona únicamente para comparar ritmos de crecimiento; su estudio algorítmico detallado corresponde a temas posteriores (Cormen et al., 2022; Sedgewick y Wayne, 2011; Knuth, 1976).

El crecimiento lineal aparece cuando el trabajo es proporcional al número de elementos. Examinar una vez cada posición produce expresiones como $an + b$ y se representa introductoramente mediante $O(n)$. La búsqueda del máximo es lineal porque, después de inicializar con el primer valor, compara cada uno de los $n - 1$ elementos restantes exactamente una vez (Cormen et al., 2022; Goodrich et al., 2013; Skiena, 2020).

El crecimiento cuadrático aparece con frecuencia cuando, para cada uno de n elementos, se realiza otro recorrido de n elementos. El producto $n \cdot n$ produce n^2 ejecuciones del cuerpo interno. Duplicar n hace que ese componente pase aproximadamente a ser cuatro veces mayor, diferencia que se vuelve decisiva para entradas grandes (Cormen et al., 2022; Sedgewick y

Wayne, 2011; Skiena, 2020).

Tabla 3: Comparación introductoria de funciones de crecimiento

n	1	$\log_2 n$	n	n^2
10	1	3.32	10	100
100	1	6.64	100	10 000
1 000	1	9.97	1 000	1 000 000

3.12.5. Relación introductoria con el análisis asintótico

La función exacta conserva constantes y términos de menor grado producidos por el criterio de conteo. El análisis asintótico observa qué término domina cuando n crece. Por ejemplo, $6n^2 + 5n + 3$ es cuadrática porque n^2 termina dominando a n y a la constante. La notación no afirma que los otros términos sean inexistentes, sino que no modifican la clase principal de crecimiento (Cormen et al., 2022; Knuth, 1976; Sedgewick y Wayne, 2011).

En esta semana basta con reconocer y comparar los patrones constante, logarítmico, lineal y cuadrático. Una formalización completa de cotas superiores, inferiores y ajustadas requiere definiciones que se desarrollan posteriormente. Sin adelantarlas innecesariamente, puede afirmarse que el recorrido del máximo es $O(n)$ y el bloque doble es $O(n^2)$ bajo el modelo adoptado (Zanabria Gálvez, 2026; Cormen et al., 2022; Knuth, 1976).

La clase de crecimiento no sustituye la corrección ni todos los detalles prácticos. Dos algoritmos lineales pueden tener constantes diferentes, y un algoritmo cuadrático puede ser suficiente para una entrada pequeña. Sin embargo, el crecimiento permite prever escalabilidad: al aumentar considerablemente n , la estructura del algoritmo suele importar más que pequeñas diferencias de velocidad del equipo (Sedgewick y Wayne, 2011; Skiena, 2020; Cormen et al., 2022).

El conteo exacto solo tiene sentido acompañado por su convención. Tratar un `for` como una unidad produce una expresión distinta de expandir inicialización, condición e incremento; contar accesos a memoria también agrega términos. Lo metodológicamente correcto es declarar el modelo, aplicar la misma granularidad a los algoritmos comparados y separar la fórmula exacta de la clase de crecimiento (Cook y Reckhow, 1973; Aho et al., 1974; Cormen et al., 2022).

El tamaño n funciona como puente entre programa y función matemática. Para el máximo representa elementos; en los ciclos anidados sigue representando el límite de cada recorrido. Una vuelta completa genera un término lineal, mientras repetirla para cada valor exterior produce el producto $n \cdot n$. La fórmula conserva el detalle del conteo y el término dominante resume la tendencia al crecer la entrada (Cormen et al., 2022; Sedgewick

y Wayne, 2011; Knuth, 1976).

La conclusión integradora es que corrección y costo contestan preguntas diferentes pero complementarias. El invariante demuestra que `encontrar_maximo` devuelve el valor debido; el modelo RAM muestra que lo consigue con crecimiento lineal. En el fragmento anidado, el conteo explica por qué el trabajo dominante es cuadrático, independientemente de que el equipo ejecute cada operación con mayor o menor rapidez (Hoare, 1969; Cook y Reckhow, 1973; Skiena, 2020).

4. Desarrollo práctico

4.1. Ficha diagnóstica razonada

1. **¿Qué es un algoritmo?** Es un conjunto finito, ordenado y no ambiguo de pasos que transforma una entrada en una salida para resolver un problema definido. Un ejemplo cotidiano es seguir una ruta: se parte de una ubicación, se ejecutan giros y desplazamientos precisos y se termina en el destino. Cumple la definición si las indicaciones son efectivas, están determinadas y concluyen.
2. **Propiedades.** La finitud exige terminación; la definición precisa elimina ambigüedades; la entrada indica los valores iniciales; la salida determina el resultado; y la efectividad exige pasos básicos que puedan ejecutarse en tiempo finito.
3. **Primitivo frente a TAD.** Un tipo primitivo es una categoría básica ya definida por el lenguaje, como un entero o un booleano. Un TAD especifica un conjunto de valores y operaciones con reglas propias, como una pila, y puede admitir varias implementaciones internas.
4. **Corrección.** Un algoritmo es correcto si satisface la especificación para toda entrada válida. La corrección parcial garantiza el resultado en caso de terminación; la total añade una prueba de que el algoritmo termina.
5. **Experiencia del grupo.** El grupo posee experiencia introductoria en variables, condicionales, ciclos, funciones, arreglos y compilación básica. Debe reforzar la formulación de invariantes, la demostración formal y el conteo de operaciones implícitas, porque estos aspectos requieren explicar no solo qué hace el código, sino por qué funciona y cuánto trabajo realiza.
6. **Fragmento de ciclos anidados.** El cuerpo se ejecuta n^2 veces. El conteo exacto cambia si se agrupan o expanden los controles del ciclo. En la Sección 7.2 se adopta un criterio explícito y se obtiene la fórmula por categorías.

5. Implementación del algoritmo para encontrar el máximo

5.1. Idea de solución

El algoritmo conserva el mayor valor observado. Inicialmente solo se ha observado el primer elemento, por lo que este es el máximo provisional. Después se recorren las posiciones restantes. Si el elemento actual es mayor, reemplaza al máximo; si no, se conserva el valor

anterior. Al finalizar se han considerado todos los elementos.

El arreglo común de prueba contiene 24 enteros:

$$A = [34, -7, 12, 56, 23, 91, 4, 18, 67, 45, 0, 39, 72, 11, 8, 63, 27, 50, 16, 5, 88, 31, -12, 70].$$

5.2. Implementación en Python

Listing 1: Implementación completa y comentada en Python

```

1  """Implementacion comentada del algoritmo encontrar_maximo para SIS210.""" # Describe
   el proposito del archivo.
2
3
4  def encontrar_maximo(arr): # Define una funcion que recibe una secuencia no vacia.
5      """Devuelve el mayor elemento de arr mediante un recorrido lineal.""" # Documenta
   el resultado de la funcion.
6      if len(arr) == 0: # Comprueba si la secuencia incumple la precondition de no
   estar vacia.
7          raise ValueError("El arreglo no puede estar vacio") # Informa claramente el
   error si no hay elementos.
8      maximo = arr[0] # Toma el primer elemento como maximo provisional.
9      for i in range(1, len(arr)): # Recorre los indices desde el segundo elemento
   hasta el ultimo.
10         if arr[i] > maximo: # Compara el elemento actual con el mayor encontrado
   hasta ahora.
11             maximo = arr[i] # Actualiza el maximo solamente cuando aparece un valor
   mayor.
12     return maximo # Devuelve el mayor valor una vez revisado todo el arreglo.
13
14
15  datos = [34, -7, 12, 56, 23, 91, 4, 18, 67, 45, 0, 39, 72, 11, 8, 63, 27, 50, 16, 5,
   88, 31, -12, 70] # Crea el arreglo de prueba con 24 enteros.
16  resultado = encontrar_maximo(datos) # Ejecuta el algoritmo desarrollado y guarda su
   respuesta.
17  maximo_real = max(datos) # Obtiene un valor de referencia con la funcion estandar de
   Python.
18  coinciden = resultado == maximo_real # Verifica si el resultado calculado coincide
   con la referencia.
19  print(f"Arreglo ({len(datos)} elementos): {datos}") # Muestra los datos utilizados en
   la prueba.
20  print(f"Maximo calculado: {resultado}") # Muestra el valor encontrado por el
   algoritmo.

```

```

21 print(f"Maximo de referencia: {maximo_real}") # Muestra el valor obtenido mediante la
    biblioteca estandar.
22 print(f"Verificacion: {'CORRECTA' if coinciden else 'INCORRECTA'}") # Comunica si
    ambas respuestas son iguales.

```

5.2.1. Explicación por bloques

- **Definición y validación:** la función recibe `arr`; si su longitud es cero, genera una excepción para hacer explícita la precondition.
- **Inicialización:** asigna el primer elemento a `maximo`. En ese instante, el máximo del prefijo de un elemento ya es conocido.
- **Recorrido:** el `for` comienza en el índice 1. Cada elemento se compara con el máximo provisional y solo se actualiza cuando es mayor.
- **Retorno y prueba:** la función devuelve el resultado. El programa lo contrasta con `max(datos)` y muestra si ambos coinciden.

5.3. Implementación en C++

Listing 2: Implementación completa y comentada en C++

```

1 #include <algorithm> // Permite usar max_element para obtener el valor de referencia.
2 #include <iostream> // Permite mostrar informacion en la consola.
3 #include <stdexcept> // Permite lanzar invalid_argument ante un arreglo vacio.
4 #include <vector> // Permite almacenar los numeros en un vector dinamico.
5
6 int encontrarMaximo(const std::vector<int>& arr) { // Recibe el vector por referencia
    constante y devuelve su maximo.
7     if (arr.empty()) { // Comprueba si el vector incumple la precondition de no estar
        vacio.
8         throw std::invalid_argument("El arreglo no puede estar vacio"); // Detiene la
            funcion con un mensaje explicativo.
9     } // Finaliza la validacion de la entrada.
10    int maximo = arr[0]; // Toma el primer elemento como maximo provisional.
11    for (std::size_t i = 1; i < arr.size(); ++i) { // Recorre el vector desde el
        segundo elemento hasta el ultimo.
12        if (arr[i] > maximo) { // Compara el elemento actual con el mayor encontrado
            hasta ese momento.
13            maximo = arr[i]; // Actualiza el maximo cuando el elemento actual es
                mayor.
14        } // Finaliza la condicion de actualizacion.
15    } // Finaliza el recorrido lineal del vector.
16    return maximo; // Devuelve el mayor valor despues de revisar todos los elementos.

```

```

17 } // Finaliza la funcion encontrarMaximo.
18
19 int main() { // Inicia el programa de prueba.
20     const std::vector<int> datos = {34, -7, 12, 56, 23, 91, 4, 18, 67, 45, 0, 39, 72,
21     11, 8, 63, 27, 50, 16, 5, 88, 31, -12, 70}; // Crea el vector de prueba con 24
22     enteros.
23     const int resultado = encontrarMaximo(datos); // Ejecuta el algoritmo
24     desarrollado.
25     const int maximoReal = *std::max_element(datos.begin(), datos.end()); // Calcula
26     la referencia con la biblioteca estandar.
27     const bool coinciden = resultado == maximoReal; // Verifica que los dos valores
28     sean iguales.
29     std::cout << "Arreglo (" << datos.size() << " elementos): ["; // Inicia la
30     impresion del vector y de su tamano.
31     for (std::size_t i = 0; i < datos.size(); ++i) { // Recorre los elementos
32     exclusivamente para mostrarlos.
33         std::cout << datos[i]; // Imprime el elemento ubicado en la posicion actual.
34         if (i + 1 < datos.size()) { // Comprueba si todavia quedan elementos por
35         imprimir.
36             std::cout << ", "; // Separa los elementos mediante una coma y un espacio.
37
38         } // Finaliza la condicion usada para separar los elementos.
39     } // Finaliza el recorrido de impresion.
40     std::cout << "]\n"; // Cierra la representacion del vector y cambia de linea.
41     std::cout << "Maximo calculado: " << resultado << '\n'; // Muestra el valor
42     producido por encontrarMaximo.
43     std::cout << "Maximo de referencia: " << maximoReal << '\n'; // Muestra el valor
44     obtenido con max_element.
45     std::cout << "Verificacion: " << (coinciden ? "CORRECTA" : "INCORRECTA") << '\n';
46     // Comunica el resultado de la comprobacion.
47     return coinciden ? 0 : 1; // Termina con exito solo cuando ambos resultados
48     coinciden.
49 } // Finaliza la funcion principal.

```

5.3.1. Explicación por bloques

- **Bibliotecas:** se importan las herramientas para vectores, salida, excepciones y la verificación con `max_element`.
- **Función:** el vector se recibe mediante referencia constante para no copiarlo ni modificarlo. La entrada vacía produce una excepción.
- **Recorrido:** una variable de tipo `std::size_t` recorre índices válidos y se aplica la misma comparación que en Python.

- **Programa principal:** crea los 24 valores, ejecuta el algoritmo, calcula una referencia independiente y termina con código cero cuando la verificación es correcta.

6. Análisis de corrección de encontrar_maximo

6.1. Explicación comprensible

La función no intenta conocer el máximo desde el inicio. Mantiene una respuesta provisional que es correcta para la parte ya recorrida. Empieza con el primer elemento; después, al mirar un nuevo valor, solo existen dos posibilidades: el nuevo valor supera al máximo anterior y debe reemplazarlo, o no lo supera y el máximo anterior sigue siendo válido. Al repetir este razonamiento hasta el final, la respuesta provisional se convierte en el máximo de todo el arreglo.

6.2. Formalización del problema

Sea A un arreglo de longitud n .

Problema: encontrar el mayor valor almacenado en A .

Entrada: arreglo finito $A[0 \dots n - 1]$ con elementos comparables.

Salida: un valor m que pertenece a A y no es menor que ningún elemento.

Precondición P : $n \geq 1$ y la relación de comparación entre elementos está definida.

Postcondición Q : $m \in A \wedge \forall k (0 \leq k < n \Rightarrow A[k] \leq m)$.

La tripleta de Hoare es:

$$\{P\} \quad S_{\max} \quad \{Q\}, \quad P : n \geq 1 \wedge A[0 \dots n - 1] \text{ contiene elementos comparables}, \\ Q : m \in A \wedge \forall k \in [0, n - 1], A[k] \leq m.$$

6.3. Invariante y demostración

Invariante: antes de cada iteración cuyo índice es i , con $1 \leq i \leq n$, la variable `maximo` es el máximo del prefijo $A[0 \dots i - 1]$.

6.3.1. Inicio

Antes de la primera iteración se tiene $i = 1$ y se ha ejecutado `maximo = A[0]`. El prefijo $A[0 \dots 0]$ contiene un solo elemento y su máximo es $A[0]$. Por tanto, el invariante es verdadero

antes de comenzar el ciclo.

6.3.2. Preservación

Se supone que antes de una iteración el invariante es verdadero, de modo que `maximo` es el máximo de $A[0 \dots i - 1]$. Si $A[i] > \text{maximo}$, se asigna $A[i]$ y el nuevo valor es el máximo de $A[0 \dots i]$. Si la condición es falsa, el máximo anterior también es mayor o igual que $A[i]$ y continúa siendo el máximo del prefijo ampliado. Después se incrementa i , de modo que el invariante queda establecido para la siguiente iteración.

6.3.3. Finalización

El ciclo termina cuando $i = n$. Al sustituir este valor en el invariante, `maximo` es el máximo de $A[0 \dots n - 1]$, es decir, de todo el arreglo. Esta afirmación implica la postcondición. Además, $n - i$ es inicialmente finito, disminuye en uno en cada repetición y no puede disminuir indefinidamente sin llegar a cero. Por ello el ciclo termina.

6.3.4. Conclusión de la demostración

El inicio y la preservación demuestran la corrección parcial: si el ciclo finaliza, el resultado satisface la postcondición. El argumento mediante $n - i$ demuestra la terminación. La combinación de ambos establece la corrección total de `encontrar_maximo` para cualquier arreglo que satisfaga la precondición.

7. Análisis mediante el modelo RAM

7.1. Criterio adoptado

Para que el conteo sea reproducible, se asigna costo unitario a cada asignación, comparación, suma y multiplicación. Los controles de los ciclos se expanden en inicialización, condición e incremento. El acceso al arreglo y el retorno se reconocen, pero no se suman como categorías independientes porque el enunciado concentra el análisis en las cuatro categorías anteriores. Si se adoptara otra granularidad, cambiaría la fórmula exacta, no el crecimiento dominante.

7.2. Dos ciclos anidados

El fragmento analizado es:

Listing 3: Fragmento original

```
1 s = 0
```

```

2 for i in range(n):
3     for j in range(n):
4         s = s + i * j

```

Una representación equivalente que hace visibles las operaciones de control es:

Listing 4: Representación equivalente con ciclos while

```

1 s = 0
2 i = 0
3 while i < n:
4     j = 0
5     while j < n:
6         s = s + i * j
7         j = j + 1
8     i = i + 1

```

7.2.1. Por qué el cuerpo se ejecuta n^2 veces

El ciclo exterior toma n valores de i : $0, 1, \dots, n-1$. Para cada uno de ellos, el ciclo interior vuelve a comenzar en cero y toma n valores de j . Por tanto, existen n ejecuciones interiores por cada una de las n ejecuciones exteriores:

$$\underbrace{n + n + \dots + n}_{n \text{ veces}} = n \cdot n = n^2.$$

También puede verse como el conjunto de pares (i, j) . Para $n = 3$ se procesan nueve pares: $(0, 0), (0, 1), (0, 2), (1, 0), \dots, (2, 2)$. Generalizar las tres filas y tres columnas a n filas y n columnas produce n^2 pares.

7.2.2. Conteo por categorías

Tabla 4: Conteo expandido del fragmento con ciclos anidados

Operación	Frecuencia	Justificación
Asignar $s = 0$	1	Se ejecuta antes de los ciclos.
Asignar $i = 0$	1	Inicialización del ciclo exterior.
Comparar $i < n$	$n + 1$	Una vez por iteración y una comprobación final falsa.

Operación	Frecuencia	Justificación
Asignar $j = 0$	n	Se reinicia una vez por cada valor de i .
Comparar $j < n$	$n(n+1) = n^2 + n$	Para cada una de las n vueltas exteriores hay n verdaderas y una falsa.
Multiplicar $i \cdot j$	n^2	Una vez por cada par (i, j) .
Sumar $s + (i \cdot j)$	n^2	Una vez dentro de cada ejecución del cuerpo.
Asignar el nuevo valor a s	n^2	La expresión actualiza s en cada par.
Sumar $j + 1$	n^2	Incremento interior en cada ejecución.
Asignar el nuevo valor a j	n^2	Almacena cada incremento interior.
Sumar $i + 1$	n	Incremento al terminar cada recorrido interior.
Asignar el nuevo valor a i	n	Almacena cada incremento exterior.

Agrupando por categoría:

$$A(n) = 1 + 1 + n + n^2 + n^2 + n = 2n^2 + 2n + 2,$$

$$C(n) = (n + 1) + (n^2 + n) = n^2 + 2n + 1,$$

$$S(n) = n^2 + n^2 + n = 2n^2 + n,$$

$$M(n) = n^2.$$

La suma de las cuatro categorías es:

$$T(n) = A(n) + C(n) + S(n) + M(n) = 6n^2 + 5n + 3.$$

Cada término tiene un origen identificable. Los $6n^2$ reúnen dos asignaciones por ejecución interior, una comparación interior dominante, dos sumas y una multiplicación. Los $5n$ proceden de reinicios e incrementos exteriores y de las comprobaciones finales interiores. La constante 3 corresponde a las dos inicializaciones previas y a la comprobación final adicional del ciclo exterior. Esta fórmula solo es exacta bajo el criterio declarado.

Si se cuentan únicamente las instrucciones visibles del cuerpo y se trata cada `for` como una construcción de control agrupada, se informaría una inicialización de s , n^2 multiplicaciones, n^2 sumas y n^2 asignaciones a s . Si además solo se consideran las comparaciones solicitadas de manera explícita, podría aparecer otra expresión. La diferencia no es una contradicción: responde a niveles distintos de descomposición. En todos los casos, el término dominante es proporcional a n^2 .

7.3. Conteo de `encontrar_maximo`

Para hacer comparable el análisis, se representa el recorrido como:

Listing 5: Búsqueda del máximo con control explícito

```
1  maximo = A[0]
2  i = 1
3  while i < n:
4      if A[i] > maximo:
5          maximo = A[i]
6      i = i + 1
7  return maximo
```

El ciclo tiene $n - 1$ iteraciones. La comparación $A[i] > \text{maximo}$ se realiza $n - 1$ veces. En el peor caso para las asignaciones, los elementos aparecen en orden estrictamente creciente y la actualización de `maximo` también ocurre $n - 1$ veces. Si se consideran solamente las operaciones principales sobre los datos, hay $n - 1$ comparaciones y $1 + (n - 1) = n$ asignaciones relacionadas con el máximo.

Con el control expandido aparecen además una asignación inicial de i , n comparaciones $i < n$, $n - 1$ sumas de incremento y $n - 1$ asignaciones de incremento. Por tanto, bajo el criterio completo:

$$A_{\max}(n) = 1 + 1 + (n - 1) + (n - 1) = 2n,$$

$$C_{\max}(n) = n + (n - 1) = 2n - 1,$$

$$S_{\max}(n) = n - 1.$$

Todas estas funciones crecen linealmente. A diferencia del fragmento anterior, no existe un segundo recorrido completo por cada elemento; por ello no aparece un término n^2 .

8. Resultados

Las pruebas se ejecutaron el 9 de septiembre de 2026 con Python 3.12.14 y g++ 13.3.0 en modo C++17. La compilación se realizó con `-Wall`, `-Wextra` y `-Wpedantic`; no se produjeron advertencias. Las dos salidas se compararon y no presentaron diferencias.

Tabla 5: Resultados de ejecución

Lenguaje	Cantidad de elementos	Máximo calculado	Verificación
Python	24	91	CORRECTA
C++	24	91	CORRECTA

El resultado real del arreglo es 91. En Python se contrastó con `max(datos)` y en C++ con `std::max_element`. Estas funciones se usaron exclusivamente como referencia de prueba; el resultado principal fue obtenido por las implementaciones desarrolladas.

9. Comparación y discusión

Tabla 6: Comparación de los algoritmos analizados

Aspecto	Encontrar máximo	Ciclos anidados
Estructura	Un recorrido desde el segundo elemento	Un recorrido completo dentro de otro
Ejecuciones principales	$n - 1$ comparaciones de datos	n^2 ejecuciones del cuerpo
Crecimiento dominante	Lineal, $O(n)$	Cuadrático, $O(n^2)$
Efecto de duplicar n aproximado	Duplica el trabajo dominante	Cuadruplica el trabajo dominante

La comparación muestra que una diferencia estructural pequeña en el código puede generar una diferencia grande al crecer la entrada. Ambos fragmentos son finitos y sus operaciones son efectivas, pero recorren espacios de tamaños distintos. Esta observación justifica analizar el costo mediante n y no depender solamente de una medición puntual.

También se confirma que corrección y eficiencia son propiedades distintas. La demostración del máximo garantiza su resultado y terminación; el conteo muestra además que lo obtiene con crecimiento lineal. Si se propusiera otro algoritmo correcto que comparase repetidamente los mismos elementos, la corrección podría conservarse mientras empeora el costo.

10. Dificultades encontradas y aprendizaje

La primera dificultad fue comprender que el modelo RAM no describe exactamente un procesador particular, sino una máquina ideal utilizada para comparar algoritmos. La dificultad se resolvió separando tiempo físico y número de operaciones: una computadora puede ser más rápida que otra, pero ambas ejecutan el mismo patrón de n o n^2 pasos bajo el criterio abstracto.

La segunda dificultad fue contar las operaciones implícitas de un `for`. La sintaxis oculta la inicialización, la comparación que permite continuar y el incremento. Reescribir los ciclos mediante `while` hizo visibles estas acciones y permitió justificar cada término del conteo, incluida la comparación final falsa.

La tercera dificultad fue comprender los invariantes. Al inicio podían parecer afirmaciones demasiado formales. Se entendieron al interpretarlos como el significado de las variables durante un avance parcial: `maximo` no representa desde el primer instante el máximo global, sino el máximo de la parte que ya se revisó.

Finalmente, fue necesario distinguir corrección parcial y total. La corrección parcial estudia el resultado suponiendo que el proceso termina; la total agrega una prueba de terminación. El ejemplo de un índice que no se incrementa mostró que una propiedad puede preservarse y, aun así, el programa quedar atrapado en un ciclo.

11. Aspectos que resultaron interesantes

Un aspecto central es que un algoritmo correcto no necesariamente es eficiente. La especificación y su demostración determinan si la respuesta es válida, mientras el análisis de costo estudia los recursos utilizados. Evaluar solamente una de estas dimensiones deja incompleta la valoración de una solución.

También resulta importante que el costo pueda estudiarse sin depender de la velocidad exacta de una computadora. El modelo RAM abstrae diferencias de hardware y permite observar la estructura del procedimiento. Esto hace posible comparar dos propuestas antes de implementarlas en todos los equipos posibles.

La diferencia entre un recorrido y dos ciclos anidados se vuelve clara al pensar en elementos y pares. Un recorrido procesa aproximadamente n posiciones; los ciclos anidados procesan n^2 pares. Al duplicar la entrada, el primero aproximadamente duplica su trabajo y el segundo lo cuadruplica.

Finalmente, la demostración formal muestra que la confianza en un algoritmo no tiene que depender solo de ejemplos. La precondition aclara cuándo puede aplicarse; el invariante explica por qué cada paso conserva el progreso; y la finalización conecta ese progreso con la

respuesta requerida.

12. Conclusiones

1. Un algoritmo requiere una entrada y una salida definidas, pasos precisos y efectivos, y terminación finita; estas propiedades hacen posible implementarlo y analizarlo, aunque todavía se necesita probar su corrección.
2. Los TAD separan las operaciones observables de los detalles internos. Esta separación permite implementar una misma abstracción con estructuras diferentes sin modificar el código usuario cuando se conserva el contrato.
3. La tripleta de Hoare organiza la relación entre supuestos, programa y resultado. Para `encontrar_maximo`, el invariante establece que el máximo provisional corresponde siempre al prefijo ya procesado.
4. El inicio, la preservación y la finalización prueban la corrección parcial; el descenso de $n - i$ demuestra terminación. En conjunto se obtiene corrección total para arreglos no vacíos de elementos comparables.
5. Bajo el conteo expandido adoptado, los ciclos anidados producen $T(n) = 6n^2 + 5n + 3$, mientras las categorías de `encontrar_maximo` permanecen lineales. La fórmula exacta depende del criterio, pero los crecimientos cuadrático y lineal no cambian.
6. Las implementaciones en Python y C++ se ejecutaron sobre 24 elementos y ambas devolvieron 91. La coincidencia con referencias estándar y entre lenguajes aporta evidencia experimental consistente con la demostración general.

13. Referencias bibliográficas

Referencias

- Aho, A. V., Hopcroft, J. E., & Ullman, J. D. (1974). *The design and analysis of computer algorithms*. Addison-Wesley.
- Apt, K. R., de Boer, F. S., & Olderog, E.-R. (2009). *Verification of sequential and concurrent programs* (3.^a ed.). Springer. <https://doi.org/10.1007/978-1-84882-745-5>
- Cook, S. A., & Reckhow, R. A. (1973). Time bounded random access machines. *Journal of Computer and System Sciences*, 7(4), 354–375. [https://doi.org/10.1016/S0022-0000\(73\)80029-7](https://doi.org/10.1016/S0022-0000(73)80029-7)
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4.^a ed.). MIT Press. <https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/>
- Dijkstra, E. W. (1975). Guarded commands, nondeterminacy and formal derivation of programs. *Communications of the ACM*, 18(8), 453–457. <https://doi.org/10.1145/360933.360975>
- Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2013). *Data structures and algorithms in Python*. Wiley. <https://www.wiley.com/en-us/Data+Structures+and+Algorithms+in+Python-p-9781118290279>
- Goodrich, M. T., Tamassia, R., & Mount, D. M. (2011). *Data structures and algorithms in C++* (2.^a ed.). Wiley. <https://www.wiley.com/en-us/Data+Structures+and+Algorithms+in+C++2nd+Edition-p-9781118136638>
- Hoare, C. A. R. (1969). An axiomatic basis for computer programming. *Communications of the ACM*, 12(10), 576–580, 583. <https://doi.org/10.1145/363235.363259>
- Knuth, D. E. (1976). Big omicron and big omega and big theta. *ACM SIGACT News*, 8(2), 18–24. <https://doi.org/10.1145/1008328.1008329>
- Knuth, D. E. (1997). *The art of computer programming: Vol. 1. Fundamental algorithms* (3.^a ed.). Addison-Wesley.
- Liskov, B., & Zilles, S. (1974). Programming with abstract data types. *ACM SIGPLAN Notices*, 9(4), 50–59. <https://doi.org/10.1145/942572.807045>
- Python Software Foundation. (2026). *Data structures*. Python 3 documentation. <https://docs.python.org/3/tutorial/datastructures.html>

- Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4.^a ed.). Addison-Wesley Professional. <https://algs4.cs.princeton.edu/home/>
- Skiena, S. S. (2020). *The algorithm design manual* (3.^a ed.). Springer. <https://doi.org/10.1007/978-3-030-54256-6>
- Zanabria Gálvez, A. H. (2026). *Material de la Semana 1: Introducción a algoritmos y estructuras de datos, tipos abstractos de datos, corrección y modelo RAM* [Material de curso]. Universidad Nacional del Altiplano.
- Wirth, N. (1976). *Algorithms + data structures = programs*. Prentice-Hall.

A. Evidencia de ejecución

Listing 6: Salida común verificada en Python y C++

```
1 Arreglo (24 elementos): [34, -7, 12, 56, 23, 91, 4, 18,  
2 67, 45, 0, 39, 72, 11, 8, 63, 27, 50, 16, 5, 88, 31,  
3 -12, 70]  
4 Maximo calculado: 91  
5 Maximo de referencia: 91  
6 Verificacion: CORRECTA
```

B. Preguntas probables para la sustentación

1. ¿Qué diferencia existe entre un algoritmo y un programa?
2. ¿Cuáles son las cinco propiedades de un algoritmo?
3. ¿Por qué el arreglo no vacío forma parte de la precondition?
4. ¿Qué diferencia existe entre un tipo primitivo y un TAD?
5. ¿Por qué una pila puede tener varias implementaciones?
6. ¿Cómo se lee la tripleta $\{P\}S\{Q\}$?
7. ¿Cuál es el invariante de `encontrar_maximo`?
8. ¿Cómo se demuestran inicio, preservación y finalización?
9. ¿Qué diferencia existe entre corrección parcial y total?
10. ¿Por qué $n - i$ permite demostrar terminación?
11. ¿Qué operaciones se consideran elementales en el conteo adoptado?
12. ¿Por qué el cuerpo interno se ejecuta n^2 veces?
13. ¿Por qué distintas convenciones producen fórmulas exactas diferentes?
14. ¿Cuántas comparaciones de datos realiza la búsqueda del máximo?
15. ¿En qué caso se actualiza `maximo` el mayor número de veces?
16. ¿Por qué un algoritmo correcto no necesariamente es eficiente?
17. ¿Qué diferencia existe entre tiempo medido y costo teórico?
18. ¿Cómo se verificaron las implementaciones en los dos lenguajes?

C. Conceptos prioritarios para estudiar

Antes de sustentar, los integrantes deben poder explicar sin leer: las cinco propiedades de un algoritmo; la diferencia entre interfaz e implementación; la corrección parcial y total; la tripleta de Hoare; el invariante del máximo; las tres etapas de su demostración; el criterio de conteo RAM; el origen de $n - 1$ y n^2 ; y la diferencia entre corrección y eficiencia. También deben saber ejecutar ambos programas y describir la función de cada bloque.